

Cybersecurity and National Defence

Technical Part of the Group Project

A.Y. 2025–2026

1. General Rules

The technical part of the project consists in the implementation of one small cybersecurity-related tool. Each group must select one topic among those proposed in this document and develop a working solution.

The objective of the technical part is not to build a large software system, but to show that the group is able to translate a cybersecurity concept into a correct and well-structured technical artifact. For this reason, all topics are designed around a clear input, a clear output, and a command-line execution model.

All projects must be implemented in Python 3. The program must read its input from one JSON file or from a small set of JSON files, depending on the selected topic, and must produce an output file in JSON format. The expected execution format is the following:

```
python main.py --input input_path --output output.json
```

In this command, `input_path` may indicate either a single JSON file or a directory containing multiple JSON files, according to the structure defined for the selected topic.

1.1. Files to be submitted

Each submission must include at least the following files:

- `main.py`: the main Python program that executes the project logic;
- `README.md`: a short text file explaining what the project does and how to run it. In particular, this file must explain:
 - which version of Python is expected;
 - how to install the required libraries;
 - how to execute the program;
 - which input files are included in the example;
 - which data structures are used for the main lookup operations;
 - one edge case that is explicitly handled in the implementation;
 - one limitation of the chosen design.
- `requirements.txt`: a text file listing the external Python libraries required by the project. For example, if the project uses libraries such as `requests`, `pandas`, or `matplotlib`, these libraries must be listed in this file, one per line, so that they can be installed easily on another machine. If the project uses only the Python standard library, this file may be empty.
- One complete example of input and the corresponding output. Depending on the selected topic, the input may be a single JSON file or a small set of JSON files contained in one folder.

The submission may also include additional source files if the group prefers to organize the code into multiple modules. However, the main entry point of the project must remain clear, and the `README.md` file must explain how the program should be executed.

1.2. Python environments and installation of libraries

Some projects may require external Python libraries. When this happens, the required libraries must be listed in `requirements.txt`.

A recommended way to run the project is to create a *virtual environment*. A virtual environment is an isolated Python environment used only for that project. Its purpose is to avoid conflicts with library versions already installed on the computer and to make the project easier to reproduce on another machine.

A typical sequence of commands for creating a new virtual environment, in a folder named `.venv`, is the following. Go to the project directory (where `main.py` is) and run the command:

```
python -m venv .venv
```

To enter the virtual environment, you must activate it.

On Linux or macOS, this is typically done with (supposing that you are in the project directory where you have created the `.venv`):

```
source .venv/bin/activate
```

On Windows, this is typically done with:

```
.venv\Scripts\activate
```

After activation, the required libraries can be installed with:

```
pip install -r requirements.txt
```

Then the program can be executed, for example, with:

```
python main.py --input input_path --output output.json
```

When the work is finished, the virtual environment can be left with the command:

```
deactivate
```

The creation and use of a virtual environment are strongly recommended, especially when external libraries are needed. However, the virtual environment itself is not part of the submission. What is required is that the project can be installed and executed clearly by following the instructions written in the `README.md` file.

1.3. Evaluation principles

The project will be evaluated mainly on correctness, clarity, robustness, and coherence with the topic specifications. Simplicity is appreciated when it is accompanied by correctness and good design choices. The mandatory version of each project must implement only the features explicitly required in this document. Additional features may be developed, but they will not compensate for missing mandatory requirements.

The submitted project should be reproducible on another machine. For this reason, the code, the example input files, and the installation instructions must be sufficiently clear and complete.

2. Topic A – Authorization Policy Engine

2.1. Objective

In this project, the group shall implement a simple authorization engine. The program receives a set of subjects, a set of resources, a set of authorization policies, and a set of access requests. For each request, the program must decide whether the requested action is permitted or denied. The objective is not to reproduce a complete industrial authorization framework. The objective is to model, in a clear and correct way, the basic reasoning of access control.

The engine shall follow the following rule: a request is permitted only if at least one policy explicitly permits it. If no policy matches, the request must be denied. This is therefore a default deny model.

2.2. Terminology

A few terms are used throughout the project.

A *subject* is the entity asking to perform an action (e.g., user, device, application).

A *resource* is the entity on which the action is requested, also called *object* in the literature. A resource may be, for example, a file, a grade record, or a document.

An *action* is the requested operation, such as *create*, *read*, *update* (write), or *delete*.

A *policy* is a rule that specifies under which conditions a subject may perform one or more actions on a resource.

A *request* is a concrete access attempt that must be evaluated by the engine.

2.3. Simplifying assumptions

To keep the project clear and comparable across groups, the mandatory version uses the following assumptions.

Policies are only of type *permit*. There are no explicit “deny” rules in the mandatory version.

Missing conditions in a policy mean “this field is not constrained by the policy”. For example, a request is executed at a certain time; if the policy does not place any constraints on the time, the field specifying the time can be ignored.

If a request refers to an unknown subject or to an unknown resource, the request must be denied.

Policies are evaluated in the order in which they appear in the input. In the mandatory version, it is sufficient to return the first policy that matches.

2.4. JSON model

2.4.1 subjects.json

This file defines the known subjects.

```
{
  "subjects": [
    {
      "id": "u100",
      "role": "student",
      "clearance": 1
    },
  ],
}
```

```

{
  "id": "u200",
  "role": "professor",
  "clearance": 2
},
{
  "id": "u300",
  "role": "admin",
  "clearance": 3
},
{
  "id": "u400",
  "role": "researcher",
  "clearance": 2
}
]
}

```

Each subject has:

- **id**: unique identifier;
- **role**: role of the subject;
- **clearance**: integer clearance level.

Only these three fields are mandatory in the minimum version.

2.4.2 resources.json

This file defines the known resources.

```

{
  "resources": [
    {
      "id": "g501",
      "type": "grade",
      "owner": "u100",
      "classification": "normal"
    },
    {
      "id": "e700",
      "type": "exam_record",
      "owner": "u200",
      "classification": "normal"
    },
    {
      "id": "d900",
      "type": "document",
      "owner": "u300",
      "classification": "restricted"
    },
    {
      "id": "r800",
      "type": "research_file",
      "owner": "u400",
      "classification": "internal"
    }
  ]
}

```

```
  ]  
}
```

Each resource has:

- **id**: unique identifier;
- **type**: type of resource;
- **owner**: subject id of the owner;
- **classification**: classification label.

Again, only these four fields are needed in the minimum version.

2.4.3 policies.json

This file defines the authorization policies.

```
{  
  "policies": [  
    {  
      "id": "P1",  
      "subject": {  
        "role": "professor"  
      },  
      "resource": {  
        "type": "exam_record"  
      },  
      "actions": ["read", "write"]  
    },  
    {  
      "id": "P2",  
      "subject": {  
        "role": "student"  
      },  
      "resource": {  
        "type": "grade",  
        "owner_is_subject": true  
      },  
      "actions": ["read"]  
    },  
    {  
      "id": "P3",  
      "subject": {  
        "role": "admin"  
      },  
      "resource": {  
        "type": "document"  
      },  
      "actions": ["delete"],  
      "context": {  
        "hour_min": 8,  
        "hour_max": 18  
      }  
    },  
    {  
      "id": "P4",
```

```

    "subject": {
      "clearance_min": 3
    },
    "resource": {
      "classification": "restricted"
    },
    "actions": ["read"],
    "context": {
      "mfa_required": true
    }
  },
  {
    "id": "P5",
    "subject": {
      "role": "researcher"
    },
    "resource": {
      "type": "research_file",
      "classification": "internal"
    },
    "actions": ["read"]
  }
]
}

```

A policy contains three main parts:

- the *subject* part defines conditions on the subject, that is, on the entity that is requesting access; for instance, a policy may apply only to subjects having role **student**, or only to subjects whose clearance level is greater than or equal to a given threshold;
- the *resource* part defines conditions on the resource, that is, on the object to which access is requested; for instance, a policy may apply only to resources of type **grade**, or only to resources whose classification is **restricted**;
- the *actions* part defines which operations are permitted when the subject and resource conditions are satisfied; for instance, a policy may allow only the action **read**, or it may allow both **read** and **write**;
- an optional *context* part may define additional constraints related to the circumstances of the request; for instance, a policy may require that **mfa** is enabled, or that the request is performed within a certain time interval.

Meaning of the policy fields The following interpretation should be stated explicitly. If a field is present in a policy, it must be satisfied. If a field is absent from a policy, it is not checked.

For the minimum project, the following condition types are enough:

- exact equality, for example `role = student`
- minimum clearance, through `clearance_min`
- ownership relation, through `owner_is_subject`
- exact context booleans, such as `mfa_required`
- hour interval, through `hour_min` and `hour_max`

2.4.4 requests.json

This file contains the access requests that must be evaluated.

```
{
  "requests": [
    {
      "request_id": "R1",
      "subject_id": "u100",
      "resource_id": "g501",
      "action": "read",
      "context": {
        "hour": 10,
        "mfa": false
      }
    },
    {
      "request_id": "R2",
      "subject_id": "u100",
      "resource_id": "e700",
      "action": "read",
      "context": {
        "hour": 10,
        "mfa": false
      }
    },
    {
      "request_id": "R3",
      "subject_id": "u300",
      "resource_id": "d900",
      "action": "delete",
      "context": {
        "hour": 21,
        "mfa": true
      }
    },
    {
      "request_id": "R4",
      "subject_id": "u300",
      "resource_id": "d900",
      "action": "read",
      "context": {
        "hour": 11,
        "mfa": true
      }
    },
    {
      "request_id": "R5",
      "subject_id": "u400",
      "resource_id": "r800",
      "action": "read",
      "context": {
        "hour": 9,
        "mfa": false
      }
    }
  ]
}
```

2.5. Exact decision logic

For each request, the program shall perform the following steps.

It searches the subject identified by `subject_id`. It searches the resource identified by `resource_id`.

If the subject does not exist, the decision is `Deny`. If the resource does not exist, the decision is `Deny`.

Then, for each policy, the program shall check whether all the following are true:

1. the subject of the request satisfies all subject conditions of the policy;
2. the resource of the request satisfies all resource conditions of the policy;
3. the requested action is contained in the policy action list;
4. the request context satisfies all context conditions of the policy.

A request is *permitted* if at least one policy matches.

If no policy matches, the request is *denied*.

2.5.1 Interpretation of selected conditions

If a policy contains:

```
"subject": { "role": "student" }
```

then the subject role must be exactly `student`.

If a policy contains:

```
"subject": { "clearance_min": 3 }
```

then the subject clearance must be greater than or equal to 3.

If a policy contains:

```
"resource": { "owner_is_subject": true }
```

then the resource owner must be equal to the subject id of the request.

If a policy contains:

```
"context": { "mfa_required": true }
```

then the request context must contain `"mfa": true`.

If a policy contains:

```
"context": { "hour_min": 8, "hour_max": 18 }
```

then the request hour must satisfy `8 <= hour <= 18`.

2.5.2 Expected output

```
{
  "summary": {
    "total_requests": 5,
    "permitted_requests": 3,
    "denied_requests": 2
  },
  "results": [
    {
      "request_id": "R1",
      "decision": "Permit",
      "matched_policy": "P2",
      "matched_conditions": [
        "subject.role=student",
        "resource.type=grade",
        "owner_is_subject=true",
        "action=read"
      ]
    },
    {
      "request_id": "R2",
      "decision": "Deny",
      "matched_policy": null,
      "matched_conditions": []
    },
    {
      "request_id": "R3",
      "decision": "Deny",
      "matched_policy": null,
      "matched_conditions": []
    },
    {
      "request_id": "R4",
      "decision": "Permit",
      "matched_policy": "P4",
      "matched_conditions": [
        "subject.clearance>=3",
        "resource.classification=restricted",
        "action=read",
        "context.mfa=true"
      ]
    },
    {
      "request_id": "R5",
      "decision": "Permit",
      "matched_policy": "P5",
      "matched_conditions": [
        "subject.role=researcher",
        "resource.type=research_file",
        "resource.classification=internal",
        "action=read"
      ]
    }
  ]
}
```

The field `matched_conditions` is intended to provide a compact explanation of why a request was permitted. In the mandatory version, denied requests may simply contain an empty list.

2.6. Implementation hint for students

1. load the four files;
2. build two dictionaries:
 - subject id $\rightarrow$ subject
 - resource id $\rightarrow$ resource
3. for each request:
 - retrieve subject and resource
 - check policies one by one
 - return the first matching policy, or deny if none matches
 - record the matched conditions for the selected policy
4. build a short summary section with the total number of permitted and denied requests.

3. Topic B – Cyber Risk Prioritization and Treatment Tool

3.1. Objective

In this project, the group shall implement a simple cyber risk prioritization and treatment tool.

The program receives a set of assets, a set of threat scenarios, and a catalog of security controls. For each scenario, the program must compute the initial risk, estimate the residual risk after the controls that are already deployed for that scenario, and determine whether the residual risk is acceptable for the affected asset.

If the residual risk is above the acceptable threshold defined for the asset, the program shall recommend one or more additional controls, selected among the controls that are known by the system, are applicable to the threat of the scenario, and are not yet deployed in that scenario.

The objective is not to reproduce a complete industrial risk management platform. The objective is to model, in a clear and correct way, the basic reasoning of asset-based cyber risk analysis and simple risk treatment.

For the purposes of this project, the following simplified model shall be used:

$$Risk = Likelihood \times Impact$$

3.2. Terminology

An *asset* is something that has value and must be protected. In this project, an asset may be, for example, a digital service, a database, a workstation, or a network gateway.

A *threat scenario* describes a possible harmful event affecting an asset. A scenario includes the affected asset, the threat, the relevant exposure, the values used to estimate the risk, and the list of controls that are already deployed for that scenario.

A *security control* is a protective measure that the risk analysis engine knows and can evaluate. A control may be preventive, detective, corrective, or protective in a broader sense. Some controls mainly reduce likelihood, because they make the attack less probable or less feasible. Other

controls mainly reduce impact, because they limit the damage if the attack succeeds or make recovery faster and more effective. A control is said to be *deployed* in a scenario if that scenario explicitly lists the identifier of the control among its deployed controls.

The *initial risk* is the risk before applying the deployed controls.

The *residual risk* is the risk after applying the deployed controls.

The *acceptable risk threshold* is the maximum residual risk tolerated for the asset.

3.3. How to interpret the scores

To keep the project simple and comparable across groups, all scores are integers from 1 to 5.

A likelihood value of 1 means that the scenario is unlikely in the considered context. A likelihood value of 5 means that the scenario is highly plausible in the considered context. An impact value of 1 means that the consequences are limited. An impact value of 5 means that the consequences are very severe.

The fields **value** and **criticality**, referred to assets, are also expressed on a 1 to 5 scale. A larger value means a more important asset. A larger criticality means more serious operational consequences in case of compromise or unavailability. The asset field **risk_threshold** indicates the maximum residual risk that is acceptable for that asset. For example, if **risk_threshold** is 6, then a residual risk of 8 is considered not acceptable and requires further treatment.

3.4. Simplifying assumptions

To keep the project clear and comparable across groups, the mandatory version uses the following assumptions.

Likelihood and impact are integer values between 1 and 5.

The initial risk is computed as the product of likelihood and impact.

A deployed control may reduce likelihood, impact, or both, according to the values explicitly specified in the control catalog.

If multiple deployed controls apply to the same scenario, their reductions are summed.

Residual likelihood and residual impact cannot become smaller than 1.

If a scenario refers to an unknown asset, the scenario must still appear in the output, but it shall be marked as invalid.

If a scenario refers to an unknown control identifier in its **deployed_controls** list, the scenario must still appear in the output, but the unknown control must be reported among the invalid references in the output.

If the residual risk is above the acceptable threshold of the asset, the program shall recommend additional controls. In the mandatory version, a greedy strategy is sufficient: the program may sort the available controls from highest total reduction to lowest total reduction and add them one by one until the threshold is met, or until no more controls are available.

A control may be recommended only if:

- it is applicable to the threat of the scenario;
- it is not already deployed in that scenario.

3.5. JSON model

3.5.1 assets.json

This file defines the assets.

```
{
  "assets": [
    {
      "id": "A1",
      "name": "student-records-db.polito.local",
      "type": "data_service",
      "value": 5,
      "criticality": 5,
      "risk_threshold": 5
    },
    {
      "id": "A2",
      "name": "mail.polito.example",
      "type": "communication_service",
      "value": 4,
      "criticality": 4,
      "risk_threshold": 8
    },
    {
      "id": "A3",
      "name": "vpn-gateway.polito.example",
      "type": "network_service",
      "value": 5,
      "criticality": 5,
      "risk_threshold": 5
    },
    {
      "id": "A4",
      "name": "cad-workstation-07.lab.local",
      "type": "endpoint",
      "value": 3,
      "criticality": 3,
      "risk_threshold": 9
    },
    {
      "id": "A5",
      "name": "research-share-07.lab.local",
      "type": "file_service",
      "value": 4,
      "criticality": 5,
      "risk_threshold": 6
    },
    {
      "id": "A6",
      "name": "campus-iot-controller.local",
      "type": "ot_service",
      "value": 4,
      "criticality": 5,
      "risk_threshold": 4
    }
  ]
}
```

Each asset has:

- **id**: unique identifier;
- **name**: identifies the specific asset, for example `vpn-gateway.polito.example`;
- **type**: identifies the general class of the asset, for example `network_service`;
- **value**: expresses the general importance of the asset for the organization;
- **criticality**: expresses how severe the operational consequences would be if the asset were compromised, unavailable, or misused;
- **risk_threshold**: maximum acceptable residual risk for the asset.

3.5.2 security_controls.json

This file defines the catalog of security controls known by the engine.

```
{
  "security_controls": [
    {
      "id": "C1",
      "name": "daily offline backups",
      "description": "Regular offline backups that allow restoration of data after encryption or destruction events.",
      "applicable_threats": ["ransomware"],
      "likelihood_reduction": 0,
      "impact_reduction": 3
    },
    {
      "id": "C2",
      "name": "endpoint protection platform",
      "description": "Endpoint protection capable of detecting and blocking malicious binaries and suspicious execution behavior.",
      "applicable_threats": ["ransomware"],
      "likelihood_reduction": 2,
      "impact_reduction": 0
    },
    {
      "id": "C3",
      "name": "operating system and application patching",
      "description": "Timely patching of systems and applications to remove known vulnerabilities commonly used by ransomware operators.",
      "applicable_threats": ["ransomware"],
      "likelihood_reduction": 2,
      "impact_reduction": 0
    },
    {
      "id": "C4",
      "name": "secure email filtering",
      "description": "Inbound email filtering that blocks suspicious senders, malicious attachments, and known phishing links.",
      "applicable_threats": ["phishing-based account compromise"],
      "likelihood_reduction": 1,
      "impact_reduction": 0
    },
    {
      "id": "C5",
```

```

    "name": "multi-factor authentication",
    "description": "An additional authentication factor that reduces the value of
stolen passwords in account compromise scenarios.",
    "applicable_threats": [
        "phishing-based account compromise",
        "unauthorized remote access"
    ],
    "likelihood_reduction": 3,
    "impact_reduction": 0
},
{
    "id": "C6",
    "name": "security awareness training",
    "description": "User training focused on recognizing phishing attempts and
suspicious credential requests.",
    "applicable_threats": ["phishing-based account compromise"],
    "likelihood_reduction": 1,
    "impact_reduction": 0
},
{
    "id": "C7",
    "name": "restrictive firewall rules",
    "description": "Network restrictions that limit administrative access to
approved hosts or management networks only.",
    "applicable_threats": ["unauthorized remote access"],
    "likelihood_reduction": 1,
    "impact_reduction": 0
},
{
    "id": "C8",
    "name": "privileged access management policy",
    "description": "Privileged account controls enforcing strong administrator
credentials and controlled use of privileged sessions.",
    "applicable_threats": ["unauthorized remote access"],
    "likelihood_reduction": 3,
    "impact_reduction": 0
},
{
    "id": "C9",
    "name": "badge-controlled office access",
    "description": "Physical access restrictions that reduce the probability that
unauthorized persons can directly reach the workstation.",
    "applicable_threats": ["unauthorized data disclosure"],
    "likelihood_reduction": 1,
    "impact_reduction": 0
},
{
    "id": "C10",
    "name": "automatic screen lock",
    "description": "Automatic workstation locking after inactivity, reducing the
time window in which an unattended workstation can be abused.",
    "applicable_threats": ["unauthorized data disclosure"],
    "likelihood_reduction": 2,
    "impact_reduction": 0
},
{
    "id": "C11",
    "name": "full-disk encryption",

```

```

    "description": "Encryption of local workstation storage so that stolen or
accessed devices do not directly expose stored files.",
    "applicable_threats": ["unauthorized data disclosure"],
    "likelihood_reduction": 0,
    "impact_reduction": 2
  },
  {
    "id": "C12",
    "name": "access logging and periodic audit",
    "description": "Logging and review of access events in order to detect and
deter unauthorized access to shared files.",
    "applicable_threats": ["unauthorized data disclosure"],
    "likelihood_reduction": 1,
    "impact_reduction": 0
  },
  {
    "id": "C13",
    "name": "least-privilege access control",
    "description": "Review and reduction of shared-folder permissions according
to the least-privilege principle, so that users retain only the access strictly
necessary for their role.",
    "applicable_threats": ["unauthorized data disclosure"],
    "likelihood_reduction": 2,
    "impact_reduction": 0
  },
  {
    "id": "C14",
    "name": "manual recovery procedure",
    "description": "Documented recovery steps that allow operators to restore
service more quickly after controller failure.",
    "applicable_threats": ["service disruption"],
    "likelihood_reduction": 0,
    "impact_reduction": 1
  },
  {
    "id": "C15",
    "name": "redundant standby controller",
    "description": "A backup controller that can take over if the primary
controller fails, reducing both outage probability and outage severity.",
    "applicable_threats": ["service disruption"],
    "likelihood_reduction": 1,
    "impact_reduction": 2
  },
  {
    "id": "C16",
    "name": "backup encryption",
    "description": "Encryption of backup media so that unauthorized access to the
copies does not directly expose their contents.",
    "applicable_threats": ["unauthorized data disclosure"],
    "likelihood_reduction": 0,
    "impact_reduction": 3
  }
]
}

```

Each security control has:

- `id`: unique identifier;
- `name`: control name;
- `description`: short explanation of the control;
- `applicable_threats`: list of threat names for which the control may be considered applicable;
- `likelihood_reduction`: non-negative integer reduction applied to likelihood;
- `impact_reduction`: non-negative integer reduction applied to impact.

3.5.3 scenarios.json

This file defines the threat scenarios.

```
{
  "scenarios": [
    {
      "id": "S1",
      "asset_id": "A1",
      "threat": "ransomware",
      "exposure": "outdated operating system",
      "likelihood": 4,
      "impact": 5,
      "deployed_controls": ["C1", "C2"]
    },
    {
      "id": "S2",
      "asset_id": "A2",
      "threat": "phishing-based account compromise",
      "exposure": "absence of multi-factor authentication",
      "likelihood": 5,
      "impact": 3,
      "deployed_controls": ["C4"]
    },
    {
      "id": "S3",
      "asset_id": "A3",
      "threat": "unauthorized remote access",
      "exposure": "weak administrator password policy",
      "likelihood": 4,
      "impact": 5,
      "deployed_controls": ["C7"]
    },
    {
      "id": "S4",
      "asset_id": "A4",
      "threat": "unauthorized data disclosure",
      "exposure": "workstation left unlocked",
      "likelihood": 3,
      "impact": 4,
      "deployed_controls": ["C9"]
    },
    {
      "id": "S5",
      "asset_id": "A5",
      "threat": "unauthorized data disclosure",
```

```

    "exposure": "excessive shared-folder permissions",
    "likelihood": 4,
    "impact": 4,
    "deployed_controls": ["C12"]
  },
  {
    "id": "S6",
    "asset_id": "A6",
    "threat": "service disruption",
    "exposure": "single point of failure in the controller",
    "likelihood": 3,
    "impact": 5,
    "deployed_controls": ["C14"]
  },
  {
    "id": "S7",
    "asset_id": "A1",
    "threat": "unauthorized data disclosure",
    "exposure": "no protection of backup media and backup contents",
    "likelihood": 3,
    "impact": 5,
    "deployed_controls": []
  }
]
}

```

Each scenario has:

- **id**: unique identifier;
- **asset_id**: identifier of the affected asset;
- **threat**: threat name;
- **exposure**: enabling condition of the scenario;
- **likelihood**: likelihood score;
- **impact**: impact score;
- **deployed_controls**: list of identifiers of the controls already deployed for that scenario.

3.6. Exact computation logic

For each scenario, the program shall perform the following steps.

It searches the asset identified by **asset_id**.

If the asset does not exist, the scenario shall be marked as invalid.

Otherwise, the program computes:

$$initial_risk = likelihood \times impact$$

Then the program shall determine all valid controls listed in the scenario field **deployed_controls**. The total reduction on likelihood is the sum of all **likelihood_reduction** values of the deployed controls. The total reduction on impact is the sum of all **impact_reduction** values of the deployed controls.

The residual values shall be computed as follows:

$$residual_likelihood = \max(1, likelihood - total_likelihood_reduction)$$

$$residual_impact = \max(1, impact - total_impact_reduction)$$

$$residual_risk = residual_likelihood \times residual_impact$$

Then the program compares the residual risk with the threshold of the asset.

If

$$residual_risk \leq risk_threshold$$

the scenario is considered acceptable.

If

$$residual_risk > risk_threshold$$

the scenario is considered not acceptable, and the program shall recommend one or more additional controls.

In the mandatory version, the set of recommendable controls for a scenario is obtained as follows:

1. consider all controls whose **applicable_threats** list contains the threat name of the scenario;
2. remove the controls that are already deployed in that scenario;
3. sort the remaining controls from highest total reduction to lowest total reduction;
4. select them greedily until the estimated residual risk becomes smaller than or equal to the threshold, or until no more controls are available.

When controls are recommended, the program shall also estimate the projected residual risk that would be obtained if the recommended controls were added. This projected value is used only as a decision-support indication. It allows the user to understand whether the proposed controls are sufficient to reach the acceptable threshold.

Finally, all valid scenarios shall be sorted from highest residual risk to lowest residual risk. Scenarios whose residual risk is above the acceptable threshold shall be placed before scenarios that are already acceptable. The field **priority** indicates the order in which the scenarios should be considered by engineers. Priority 1 means that the scenario requires the earliest attention.

3.6.1 Meaning of the output

The output of this project is not only a numerical summary.

Its purpose is to help engineers understand:

- which scenarios are currently acceptable and which are not;
- which assets require earlier attention;
- which controls are already contributing to risk reduction;
- which additional controls may be useful;
- whether the recommended controls are sufficient to reach the acceptable threshold.

For this reason, the output shall contain both:

- a short summary of the overall analysis;
- a detailed result for each scenario.

3.6.2 Output example

```
{
  "summary": {
    "total_scenarios": 7,
    "acceptable_scenarios": 3,
    "not_acceptable_scenarios": 4,
    "highest_residual_risk": 15
  },
  "risk_results": [
    {
      "scenario_id": "S3",
      "asset_id": "A3",
      "asset_name": "vpn-gateway.polito.example",
      "threat": "unauthorized remote access",
      "exposure": "weak administrator password policy",
      "initial_risk": 20,
      "deployed_controls": ["restrictive firewall rules"],
      "residual_risk": 15,
      "acceptable_threshold": 5,
      "status": "not_acceptable",
      "recommended_controls": [
        "multi-factor authentication",
        "privileged access management policy"
      ],
      "projected_risk_after_recommendation": 5,
      "treatment_result": "threshold_met",
      "priority": 1
    },
    {
      "scenario_id": "S2",
      "asset_id": "A2",
      "asset_name": "mail.polito.example",
      "threat": "phishing-based account compromise",
      "exposure": "absence of multi-factor authentication",
      "initial_risk": 15,
      "deployed_controls": ["secure email filtering"],
      "residual_risk": 12,
      "acceptable_threshold": 8,
      "status": "not_acceptable",
      "recommended_controls": [
        "multi-factor authentication",
        "security awareness training"
      ],
      "projected_risk_after_recommendation": 3,
      "treatment_result": "threshold_met",
      "priority": 2
    },
    {
      "scenario_id": "S5",
      "asset_id": "A5",
      "asset_name": "research-share-07.lab.local",
      "threat": "unauthorized data disclosure",
      "exposure": "excessive shared-folder permissions",
      "initial_risk": 16,
      "deployed_controls": ["access logging and periodic audit"],
      "residual_risk": 12,
      "acceptable_threshold": 6,
```

```

    "status": "not_acceptable",
    "recommended_controls": [
        "least-privilege access control"
    ],
    "projected_risk_after_recommendation": 4,
    "treatment_result": "threshold_met",
    "priority": 3
},
{
    "scenario_id": "S4",
    "asset_id": "A4",
    "asset_name": "cad-workstation-07.lab.local",
    "threat": "unauthorized data disclosure",
    "exposure": "workstation left unlocked",
    "initial_risk": 12,
    "deployed_controls": ["badge-controlled office access"],
    "residual_risk": 8,
    "acceptable_threshold": 9,
    "status": "acceptable",
    "recommended_controls": [],
    "projected_risk_after_recommendation": 8,
    "treatment_result": "already_acceptable",
    "priority": 4
}
]
}

```

The field `deployed_controls` lists the controls already considered in the computation of the residual risk.

The field `recommended_controls` lists the additional controls suggested by the program when the residual risk is not acceptable.

The field `projected_risk_after_recommendation` indicates the estimated residual risk that would be obtained if the recommended controls were added.

The field `treatment_result` indicates whether the proposed controls are sufficient to reach the acceptable threshold. For example:

- `already_acceptable` means that no additional control is required;
- `threshold_met` means that the recommended controls are sufficient to reach the threshold;
- `threshold_not_met` means that even after applying all recommended controls, the estimated risk would still remain above the acceptable threshold.

3.7. Implementation hint for students

A simple implementation is sufficient.

The program may:

1. load the asset, scenario, and control files;
2. build a dictionary from asset identifier to asset;
3. build a dictionary from control identifier to control;
4. for each scenario, resolve the deployed control identifiers;

5. compute initial risk and residual risk;
6. compare the residual risk with the threshold of the corresponding asset;
7. determine which additional controls are applicable to the scenario threat and not yet deployed;
8. recommend additional controls when needed;
9. estimate the projected residual risk after the recommended controls;
10. build a summary section for the whole analysis;
11. sort the scenarios and assign the priority values.

4. Topic C – Phishing Detection and Explanation Engine

4.1. Objective

In this project, the group shall implement a rule-based phishing detection and explanation engine. The program receives a set of message descriptions and, for each message, it must estimate the risk that the message is malicious.

The program must not use machine learning. The objective is to implement a simple, explainable, and reproducible decision process based on explicit rules.

For each message, the program must produce:

- a classification;
- a numerical risk score;
- the list of triggered indicators;
- a compact explanation of why each indicator was triggered.

The objective is not to reproduce a complete industrial mail-security platform. The objective is to model, in a clear and correct way, the basic reasoning of phishing detection across common communication channels.

4.2. Terminology

A *message* is a communication received by a user. In this project, a message may represent an email, an SMS message, or a social-media message.

An *indicator* is a suspicious element observed in the message, such as an urgent tone, a request for immediate action on an account, a lookalike sender domain, a suspicious link, or a dangerous attachment.

A *rule* is an explicit condition used by the program to detect one indicator. For example, a rule may check whether the subject or body contains urgency keywords, or whether a link uses the `http` scheme instead of `https`.

The *risk score* is a numerical value computed by summing the weights of the triggered indicators.

The *classification* is the final label assigned to the message.

The *evidence* of an indicator is a compact explanatory string generated by the rule from the concrete elements found in the message. For example, an urgency rule may produce an explanation stating which keywords were found in the subject and which were found in the body.

4.3. Simplifying assumptions

To keep the project clear and comparable across groups, the mandatory version uses the following assumptions.

The program shall work on a single file named `messages.json`.

The mandatory version shall support messages from the following channels: `email`, `sms`, `social`. The objective is not to create a different detection engine for each channel, but to apply a common rule-based reasoning model to all of them.

The program shall use only explicit rules and fixed weights chosen by the group.

The program shall classify each message into one of the following three classes:

- `legitimate`
- `suspicious`
- `phishing`

The exact thresholds may be chosen by the group, but they must be clearly documented in the `README.md` file.

The mandatory version may use a simplified local knowledge base defined by the group, for example:

- a small set of trusted domains or trusted sender identifiers;
- a small set of suspicious keywords;
- a small set of dangerous attachment types;
- a small set of financial or credential-related keywords.

The mandatory version is rule-based only. No machine learning technology is required and no external online service is required.

If an optional field such as `links` or `attachments` is missing, it may be treated as an empty list.

If a channel does not normally use a given feature in practice, the corresponding field may still appear in the JSON input for uniformity, but the engine may simply ignore it when not applicable.

The mandatory version may use the same core indicators across channels, with a small number of channel-specific checks where appropriate.

4.4. JSON model

4.4.1 `messages.json`

```
{
  "messages": [
    {
      "id": "M1",
      "channel": "email",
      "sender_name": "Microsoft Security",
      "sender_address": "no-reply@microsoft.com",
      "subject": "Urgent password reset required",
      "body": "Your account will be disabled today unless you reset your password immediately.",
      "links": [
```

```

        {
            "display_text": "Reset your Microsoft password",
            "actual_url": "http://security-reset-login-example.com"
        }
    ],
    "attachments": []
},
{
    "id": "M2",
    "channel": "email",
    "sender_name": "Administration Office",
    "sender_address": "office@polito.it",
    "subject": "Exam calendar update",
    "body": "The updated exam calendar is available on the official student portal.",
    "links": [
        {
            "display_text": "Student Portal",
            "actual_url": "https://students.polito.it"
        }
    ],
    "attachments": []
},
{
    "id": "M3",
    "channel": "email",
    "sender_name": "Chief Financial Officer",
    "sender_address": "cfo-payment@company-secure.net",
    "subject": "Urgent transfer before 17:00",
    "body": "Please execute the attached invoice immediately. This is confidential and time-sensitive.",
    "links": [],
    "attachments": [
        {
            "filename": "invoice.xlsx",
            "type": "spreadsheet_macro_enabled"
        }
    ]
},
{
    "id": "M4",
    "channel": "sms",
    "sender_name": "Parcel Service",
    "sender_address": "+391234567890",
    "subject": "",
    "body": "Your parcel is on hold. Confirm delivery now at http://delivery-check.example",
    "links": [
        {
            "display_text": "Confirm delivery",
            "actual_url": "http://delivery-check.example"
        }
    ],
    "attachments": []
},
{
    "id": "M5",
    "channel": "social",

```

```

    "sender_name": "IT Support Desk",
    "sender_address": "it-support-help",
    "subject": "",
    "body": "We detected a login issue on your account. Send your username and
password here to restore access.",
    "links": [],
    "attachments": []
  }
]
}

```

Each message has:

- **id**: unique identifier;
- **channel**: communication channel, for example **email**, **sms**, or **social**;
- **sender_name**: displayed sender name;
- **sender_address**: sender address, sender identifier, account name, or phone number;
- **subject**: message subject; it may be empty for channels such as SMS or social-media messages;
- **body**: main message text;
- **links**: optional list of links;
- **attachments**: optional list of attachments.

Each link has:

- **display_text**: visible text shown to the user;
- **actual_url**: actual destination URL.

Each attachment has:

- **filename**: attachment file name;
- **type**: simplified attachment type.

4.5. Minimum required indicators

The mandatory version shall detect at least some of the following indicators:

- suspicious or unknown sender;
- lookalike domain;
- urgent language;
- request for credentials;
- request for money or payment;
- suspicious link;
- non-HTTPS destination;
- dangerous attachment type;

The group may define additional indicators. For example, the following are acceptable extensions:

- authority impersonation;
- external link inconsistent with the claimed sender;
- display text / destination mismatch;
- suspicious phone number pattern;
- suspicious shortening service.

4.6. Exact decision logic

For each message, the program shall perform the following steps.

First, it shall inspect the message fields and apply a set of explicit rules.

Each triggered rule contributes a positive number of points to the total risk score.

If multiple indicators are triggered, their weights are summed.

The final classification shall be derived from the total score according to thresholds chosen by the group.

For example, a group may define:

- 0–19: legitimate
- 20–49: suspicious
- 50 and above: phishing

This is only an example. Different thresholds may be used, provided that they are documented and applied consistently.

4.6.1 Interpretation of selected indicators

For the mandatory version, the following simplified interpretations are sufficient.

Suspicious or unknown sender This indicator may be triggered if at least one of the following holds:

- for an email, the sender domain is not in a small trusted-domain set defined by the group;
- for an SMS, the sender identifier or phone number appears untrusted, malformed, or inconsistent with the claimed sender;
- for a social-media message, the sender identifier appears untrusted, generic, or inconsistent with the claimed sender name;
- the sender name claims a well-known organization but the sender address or identifier is inconsistent with it.

Lookalike domain This indicator may be triggered if the sender domain or link domain resembles a trusted domain but is not exactly equal to it. For example, `rnicrosoft.com` resembles `microsoft.com`.

The mandatory version may implement this with a simplified approach, for example:

- a small manually defined list of known lookalike patterns;
- one-character substitution rules;

- detection of common typosquatting forms chosen by the group.

Urgent language This indicator may be triggered if the subject or body contains one or more urgency keywords defined by the group, such as:

- urgent
- immediately
- today
- right now
- within 10 minutes
- account will be disabled
- time-sensitive
- confirm now
- on hold

Request for credentials This indicator may be triggered if the message asks the user to provide or confirm authentication information, such as:

- password
- credentials
- login
- username
- verification code
- account confirmation

Request for money or payment This indicator may be triggered if the message asks for a financial action, such as:

- transfer
- payment
- invoice
- bank details
- wire
- gift card

Suspicious link This indicator may be triggered if at least one link appears inconsistent with the message context. For example:

- the visible text suggests a trusted site but the actual URL points elsewhere;
- the actual destination domain is unrelated to the claimed sender;
- the link domain is clearly suspicious according to rules defined by the group.

Non-HTTPS destination This indicator may be triggered if at least one link uses the `http` scheme instead of `https`.

Dangerous attachment type This indicator may be triggered if at least one attachment type belongs to a dangerous set defined by the group, for example:

- executable
- script
- archive
- macro-enabled Office document
- unsupported or unknown attachment type

4.6.2 Channel-specific remarks

The same general reasoning model shall be applied to all channels.

A simple interpretation is sufficient:

- for **email**, students may focus on sender domain, links, and attachments;
- for **sms**, students may focus on urgency, account or parcel scams, suspicious sender identifiers, and suspicious links;
- for **social**, students may focus on impersonation, credential requests, suspicious sender identifiers, and account-recovery messages.

The objective is not to make each channel equally rich in features. The objective is to create a coherent rule-based method that can be applied to different kinds of phishing messages.

4.6.3 Evidence format for indicators

The explanation of each triggered indicator shall be generated directly by the rule that detected it.

To keep the project simple and readable, the evidence may be represented as one compact string for each triggered indicator.

This string should summarize the concrete elements that caused the rule to trigger. For example:

- “sender domain `nmicrosoft.com` resembles `microsoft.com`”;
- “urgency keywords found in subject and body: urgent, today, immediately”;
- “link uses `http` instead of `https`”;
- “attachment `invoice.xlsx` is macro-enabled.

The evidence string can be constructed programmatically from the matches found by the rule. For example, if the urgency rule checks both subject and body, the implementation may:

- collect all urgency keywords found in the subject;
- collect all urgency keywords found in the body;
- build a single explanation string from those two lists.

4.6.4 Programming example for one indicator

A simple implementation may define one function per indicator. Each function may return:

- whether the indicator was triggered;
- the indicator name;
- the indicator weight;
- one evidence string.

For example, the urgency rule may be implemented in a style similar to the following:

```
def detect_urgent_language(message):
    keywords = [
        "urgent",
        "immediately",
        "today",
        "right now",
        "within 10 minutes",
        "account will be disabled",
        "time-sensitive",
        "confirm now",
        "on hold"
    ]

    subject = message.get("subject", "").lower()
    body = message.get("body", "").lower()

    subject_matches = []
    body_matches = []

    for kw in keywords:
        if kw in subject:
            subject_matches.append(kw)
        if kw in body:
            body_matches.append(kw)

    if subject_matches or body_matches:
        parts = []

        if subject_matches:
            parts.append(
                "subject contains: " + ", ".join(subject_matches)
            )

        if body_matches:
            parts.append(
                "body contains: " + ", ".join(body_matches)
            )

        evidence = "urgency keywords found; " + "; ".join(parts)

    return {
        "triggered": True,
        "indicator": "urgent_language",
        "weight": 20,
        "evidence": evidence
    }
```

```

return {
  "triggered": False,
  "indicator": "urgent_language",
  "weight": 0,
  "evidence": ""
}

```

This is only an example. Equivalent or more sophisticated implementations are acceptable.

4.6.5 Meaning of the output

The output of this project is not only a classification label.

Its purpose is to help users understand:

- which messages appear safe and which appear dangerous;
- which suspicious indicators were detected;
- how much each indicator contributed to the total score;
- which concrete elements of the message caused the alert.

For this reason, the output shall contain both:

- a short summary of the overall analysis;
- a detailed result for each message.

4.6.6 Expected output

```

{
  "summary": {
    "total_messages": 5,
    "legitimate_messages": 1,
    "suspicious_messages": 1,
    "phishing_messages": 3,
    "highest_risk_score": 95
  },
  "results": [
    {
      "id": "M1",
      "channel": "email",
      "classification": "phishing",
      "risk_score": 95,
      "triggered_indicators": [
        {
          "name": "lookalike_domain",
          "weight": 30,
          "evidence": "sender domain rnicrosoft.com resembles microsoft.com"
        },
        {
          "name": "urgent_language",
          "weight": 20,
          "evidence": "urgency keywords found; subject contains: urgent; body contains: today, immediately"
        }
      ]
    }
  ]
}

```

```

        "name": "credential_request",
        "weight": 25,
        "evidence": "credential-related terms found in subject/body: password,
reset your password"
    },
    {
        "name": "non_https_link",
        "weight": 20,
        "evidence": "link http://security-reset-login-example.com uses http
instead of https"
    }
]
},
{
    "id": "M2",
    "channel": "email",
    "classification": "legitimate",
    "risk_score": 0,
    "triggered_indicators": []
},
{
    "id": "M3",
    "channel": "email",
    "classification": "phishing",
    "risk_score": 70,
    "triggered_indicators": [
        {
            "name": "urgent_language",
            "weight": 20,
            "evidence": "urgency keywords found; subject contains: urgent; body
contains: immediately, time-sensitive"
        }
    ],
    {
        "name": "payment_request",
        "weight": 25,
        "evidence": "payment-related terms found in subject/body: transfer,
invoice"
    },
    {
        "name": "dangerous_attachment",
        "weight": 25,
        "evidence": "attachment invoice.xlsx has dangerous type
spreadsheet_macro_enabled"
    }
]
},
{
    "id": "M4",
    "channel": "sms",
    "classification": "suspicious",
    "risk_score": 40,
    "triggered_indicators": [
        {
            "name": "urgent_language",
            "weight": 20,
            "evidence": "urgency keywords found in body: on hold, confirm now"
        }
    ],
    {

```

```

        "name": "non_https_link",
        "weight": 20,
        "evidence": "link http://delivery-check.example uses http instead of
https"
    }
],
},
{
    "id": "M5",
    "channel": "social",
    "classification": "phishing",
    "risk_score": 55,
    "triggered_indicators": [
        {
            "name": "credential_request",
            "weight": 35,
            "evidence": "credential-related terms found in body: username, password"
        },
        {
            "name": "suspicious_or_unknown_sender",
            "weight": 20,
            "evidence": "sender IT Support Desk uses untrusted identifier it-support-
help"
        }
    ]
}
]
}

```

In this example, the field `triggered_indicators` provides a structured explanation. For each indicator, it reports:

- the indicator name;
- the corresponding weight;
- one evidence string generated by the rule.

Messages classified as `legitimate` may simply contain an empty list of triggered indicators.

4.6.7 Limitations of the rule-based approach

The mandatory version of this project is intentionally rule-based. This approach has important advantages:

- the logic is transparent;
- the output is explainable;
- the behaviour is reproducible.

However, students should be aware that such a simplified approach also has limitations. For example:

- a sophisticated phishing message may evade simple keyword-based rules;
- a legitimate business message may appear urgent and therefore produce false positives;
- the quality of the result depends strongly on the chosen rules, weights, and trusted-domain or trusted-sender set.

4.7. Implementation hint for students

A simple implementation is sufficient.

The program may:

1. load the messages;
2. define a small local knowledge base, for example keywords, trusted senders, trusted domains, and dangerous attachment types;
3. define one function for each indicator;
4. let each function return:
 - whether the indicator was triggered;
 - the indicator name;
 - the indicator weight;
 - one evidence string generated from the concrete matches found by the rule;
5. execute all indicator functions on each message;
6. sum the weights of the triggered indicators;
7. determine the final class from the total score;
8. build the detailed result for each message;
9. build a short summary section for the whole analysis.

5. Topic D – Simplified PKI-based identity and signature verifier

5.1. Objective

In this project, the group shall implement a simplified PKI-based identity and signature verifier. The program receives a trust store, a certificate chain, a validation request, and, depending on the selected validation mode, an optional signed document and an optional list of revoked certificates. If the selected validation mode is `document_signature`, the program also receives a signed document. It must decide whether the end-entity certificate can be trusted for the required purpose.

The engine shall support two validation modes:

- **service_identity**: validate that a certificate chain can be trusted for a service identity, such as a portal or server (e.g., `www.polito.it`);
- **document_signature**: validate that a certificate chain can be trusted for digital signature usage and that the signature on a document is valid.

The objective is not to reproduce and verify a complete industrial PKI implementation. The objective is to model, in a clear and correct way, the basic reasoning of certificate validation, digital identity verification, and, when required, digital signature verification.

5.2. Terminology

A *certificate* is a digital document that binds an identity to a public key.

The *subject* is the identity described by the certificate.

The *issuer* is the entity that issued and signed the certificate.

A *root certificate* is a certificate that is directly trusted by the verifier.

An *intermediate certificate* is a certificate that stands between the root and the end entity.

The *end-entity certificate* is the certificate of the subject that must be validated for a specific purpose.

A *digital signature* is a cryptographic value computed using the private key of the signer over a message, or more commonly over a digest of that message.

A *document digest* is the fixed-length value obtained by applying a cryptographic hash function to the document.

The *trust store* is the local set of certification authorities that the verifier already trusts.

The *revocation list* is the list of certificate identifiers that must no longer be considered valid.

5.3. Simplifying assumptions

To keep the project clear and comparable between groups, the mandatory version uses the following assumptions.

The input is represented through JSON files.

The chain is already provided in order from leaf certificate to root certificate.

The mandatory version shall support one signature algorithm only:

- `rsa_pkcs1v15_sha256`

The public keys needed for verification are provided directly in the JSON input in PEM format.

For each certificate, the signature shall be verified over a simplified certificate payload built from the following fields only:

- `subject`
- `issuer`
- `not_before`
- `not_after`
- `key_usage`
- `is_ca`
- `public_key_pem`

This avoids the need to parse real X.509 binary structures while still requiring to perform a real signature verification step.

In `document_signature` mode, the signed document is provided as plain text in the JSON input. The program shall compute its digest and verify the corresponding signature.

Identity matching is simplified to exact equality between the expected identity and the `subject` field of the end-entity certificate.

In the mandatory version, the revocation mechanism is simplified to a local JSON list of revoked certificate identifiers.

If the chain is malformed or incomplete, the program must still produce an output that explains the reason of failure.

If the signature algorithm declared in the input is unsupported, validation must fail explicitly.

The mandatory version shall not require manual OpenSSL commands during execution. However, OpenSSL may be used to generate the example keys, signatures, and test material included in the input files.

5.3.1 What happens in reality

In real systems, digital certificates are usually represented according to the X.509 standard. They are commonly distributed in formats such as PEM or DER, and they contain several fields, including the subject, the issuer, the validity period, the public key, the intended key usage, extensions, and the digital signature.

In real deployments, certificate chains are not always provided explicitly in one input structure. For example, an end-entity certificate may contain “Authority Information Access” (AIA) references from which intermediate certificates can be downloaded, and a verifier may need to retrieve them before completing the validation process.

In real systems, digitally signed objects or documents are often represented in standard formats such as CMS, PAdES, XAdES, or CAdES.

In this project, JSON is used only for simplicity. The goal is to understand the *logic of trust verification and signature validation*, not the encoding details of industrial certificate and signature formats.

5.4. JSON model

5.4.1 trust_store.json

```
{
  "trusted_roots": [
    "RootCA1"
  ]
}
```

This file contains the list of trusted root authority names.

5.4.2 chain_www_polito.json

```
{
  "chain": [
    {
      "id": "Leaf1",
      "subject": "www.polito.it",
      "issuer": "IntermediateCA1",
      "not_before": "2026-01-01",
      "not_after": "2027-01-01",
      "key_usage": ["server_auth"],
      "is_ca": false,
      "public_key_pem": "-----BEGIN PUBLIC KEY-----\n...\n-----END PUBLIC KEY-----",
      "signature_algorithm": "rsa_pkcs1v15_sha256",
      "signature_b64": "Bd19PS0fS8..."
    },
    {
```

```

    "id": "IntermediateWeb1",
    "subject": "IntermediateCA1",
    "issuer": "RootCA1",
    "not_before": "2025-01-01",
    "not_after": "2028-01-01",
    "key_usage": ["ca"],
    "is_ca": true,
    "public_key_pem": "-----BEGIN PUBLIC KEY-----\n...\n-----END PUBLIC KEY-----",

    "signature_algorithm": "rsa_pkcs1v15_sha256",
    "signature_b64": "FQAVoRtGB/PEQQj..."
  },
  {
    "id": "Root1",
    "subject": "RootCA1",
    "issuer": "RootCA1",
    "not_before": "2020-01-01",
    "not_after": "2035-01-01",
    "key_usage": ["ca"],
    "is_ca": true,
    "public_key_pem": "-----BEGIN PUBLIC KEY-----\n...\n-----END PUBLIC KEY-----",

    "signature_algorithm": "rsa_pkcs1v15_sha256",
    "signature_b64": "cohf0023kaWSVjLJ..."
  }
]
}

```

Each certificate has:

- **id**: unique certificate identifier;
- **subject**: identity represented by the certificate;
- **issuer**: certificate issuer;
- **not_before**: validity start date;
- **not_after**: validity end date;
- **key_usage**: allowed usages;
- **is_ca**: whether the certificate is a certification authority certificate;
- **public_key_pem**: public key of the certified subject;
- **signature_algorithm**: declared signature algorithm;
- **signature_b64**: certificate signature encoded in Base64.

In the mandatory version, the certificate signature is verified over the canonical JSON serialization of the certificate payload composed of: **subject**, **issuer**, **not_before**, **not_after**, **key_usage**, **is_ca**, and **public_key_pem**.

5.4.3 signed_document.json

This file is used only in `document_signature` mode.

```

{
  "document_id": "DOC1",

```

```

    "content": "This is the content of the signed document.",
    "signature_algorithm": "rsa_pkcs1v15_sha256",
    "signature_b64": "MEQCIF..."
  }

```

This file contains:

- `document_id`: unique identifier of the document;
- `content`: the textual content of the signed document;
- `signature_algorithm`: signature algorithm used to sign the document;
- `signature_b64`: digital signature on the document content, encoded in Base64.

In the mandatory version, the signer is the subject of the first certificate in the chain.

5.4.4 `validation_request.json`

A request in `service_identity` mode may be:

```

{
  "validation_mode": "service_identity",
  "current_date": "2026-05-10",
  "expected_usage": "server_auth",
  "expected_identity": "www.polito.it"
}

```

A request in `document_signature` mode may be:

```

{
  "validation_mode": "document_signature",
  "current_date": "2026-05-10",
  "expected_usage": "digital_signature",
  "expected_identity": "Prof. Mario Rossi"
}

```

This file defines:

- `validation_mode`: either `service_identity` or `document_signature`;
- `current_date`: the date at which validation is performed;
- `expected_usage`: the expected usage of the leaf certificate;
- `expected_identity`: the identity that must be matched by the end-entity certificate.

5.4.5 `revocations.json`

```

{
  "revoked_certificates": [
    "CertX",
    "CertY"
  ]
}

```

This file contains the list of revoked certificate identifiers.

5.5. Exact validation logic

The program shall always validate the certificate chain. Then, depending on the selected validation mode, it shall perform the additional checks required for that mode.

5.5.1 Certificate-chain validation

The program shall verify the certificate chain through the following checks.

First, the chain must not be empty.

Second, for every adjacent pair of certificates in the chain, the *issuer* of the child certificate must be equal to the *subject* of the parent certificate.

Third, the subject of the last certificate in the chain must appear in the trust store.

Fourth, for every certificate in the chain, the `current_date` in the validation request must be between `not_before` and `not_after`.

Fifth, no certificate in the chain may appear in the revocation list.

Sixth, every non-leaf certificate in the chain must have `is_ca = true`.

Seventh, the leaf certificate must have `is_ca = false`.

Eighth, every certificate that issues another certificate must contain `ca` in its `key_usage` field.

Ninth, every certificate signature must be verified using the public key of its issuer. For the root certificate, a self-signature check has to be performed, but trust shall also depend on the trust store.

Tenth, the end-entity certificate must contain the expected usage in its `key_usage` field.

Eleventh, the subject of the end-entity certificate must be exactly equal to the expected identity.

5.5.2 service_identity mode

If the request specifies `validation_mode = service_identity`, the engine shall validate the certificate chain and identity of the service.

If all chain checks succeed, the leaf usage matches the requested service usage, and the leaf subject matches the expected identity, then the final decision is trusted.

No signed document is required in this mode.

5.5.3 document_signature mode

If the request specifies `validation_mode = document_signature`, the engine shall first validate the certificate chain.

Then it shall verify the digital signature on the document.

First, it shall compute the SHA-256 digest of the document content.

Second, it shall verify the document signature using:

- the public key contained in the leaf certificate;
- the algorithm declared in `signed_document.json`.

If the document signature is valid, the signer certificate is trusted, the leaf certificate is valid for the requested usage, and the subject matches the expected identity, then the signed document shall be considered valid.

5.5.4 Interpretation of the signature fields

The field `signature_algorithm` indicates which verification method must be used.

In the mandatory version, the only supported value is:

- `rsa_pkcs1v15_sha256`

If a certificate or a signed document declares a different algorithm, the corresponding signature check must fail with an explicit explanation.

The field `signature_b64` contains the signature bytes encoded using Base64.

For certificate verification, the verifier shall reconstruct the signed certificate payload by taking only the following fields:

- `subject`
- `issuer`
- `not_before`
- `not_after`
- `key_usage`
- `is_ca`
- `public_key_pem`

A simple and sufficient approach is to serialize that payload as canonical JSON with sorted keys (see example implementation in `topic-d-pki/src/signature_utils.py`).

5.5.5 Meaning of the output

The output of this project is not only a final boolean decision.

Its purpose is to help the user understand:

- whether the certificate chain is structurally correct;
- whether the certificates are trusted, valid, and not revoked;
- whether certificate signatures are valid;
- whether the requested service identity is trusted, or, in document mode, whether the document signature is valid;
- why the final decision is trusted or not trusted.

For this reason, the output shall contain both:

- a compact final summary;
- a structured explanation of the performed checks.

5.5.6 Expected output in service_identity mode

```
{
  "trusted": true,
  "validation_mode": "service_identity",
  "end_entity": "www.polito.it",
  "checks": {
    "chain_not_empty": {
      "ok": true,
      "details": "certificate chain contains 3 certificates"
    },
    "chain_continuity": {
      "ok": true,
      "details": "each child issuer matches the subject of the next certificate"
    },
    "root_trusted": {
      "ok": true,
      "details": "root subject RootCA1 is present in the trust store"
    },
    "validity_period": {
      "ok": true,
      "details": "all certificates are valid on 2026-05-10"
    },
    "revocation": {
      "ok": true,
      "details": "no certificate in the chain is revoked"
    },
    "ca_constraints": {
      "ok": true,
      "details": "intermediate and root certificates are CA certificates, leaf is not"
    },
    "certificate_signatures": {
      "ok": true,
      "details": "all certificate signatures were successfully verified"
    },
    "usage": {
      "ok": true,
      "details": "leaf certificate allows server_auth"
    },
    "identity_match": {
      "ok": true,
      "details": "leaf subject matches expected identity www.polito.it"
    }
  }
}
```

5.5.7 Expected output in document_signature mode

```
{
  "trusted": true,
  "validation_mode": "document_signature",
  "end_entity": "Prof. Mario Rossi",
  "document_id": "DOC1",
  "checks": {
    "chain_not_empty": {
```

```

    "ok": true,
    "details": "certificate chain contains 3 certificates"
  },
  "chain_continuity": {
    "ok": true,
    "details": "each child issuer matches the subject of the next certificate"
  },
  "root_trusted": {
    "ok": true,
    "details": "root subject RootCA1 is present in the trust store"
  },
  "validity_period": {
    "ok": true,
    "details": "all certificates are valid on 2026-05-10"
  },
  "revocation": {
    "ok": true,
    "details": "no certificate in the chain is revoked"
  },
  "ca_constraints": {
    "ok": true,
    "details": "intermediate and root certificates are CA certificates, leaf is not"
  },
  "certificate_signatures": {
    "ok": true,
    "details": "all certificate signatures were successfully verified"
  },
  "usage": {
    "ok": true,
    "details": "leaf certificate allows digital_signature"
  },
  "identity_match": {
    "ok": true,
    "details": "leaf subject matches expected identity Prof. Mario Rossi"
  },
  "document_signature": {
    "ok": true,
    "details": "document signature is valid under the leaf public key"
  }
}
}
}

```

An invalid case in `document_signature` mode may instead produce:

```

{
  "trusted": false,
  "validation_mode": "document_signature",
  "end_entity": "Prof. Mario Rossi",
  "document_id": "DOC1",
  "checks": {
    "chain_not_empty": {
      "ok": true,
      "details": "certificate chain contains 3 certificates"
    },
    "chain_continuity": {
      "ok": true,

```

```

    "details": "each child issuer matches the subject of the next certificate"
  },
  "root_trusted": {
    "ok": true,
    "details": "root subject RootCA1 is present in the trust store"
  },
  "validity_period": {
    "ok": true,
    "details": "all certificates are valid on 2026-05-10"
  },
  "revocation": {
    "ok": true,
    "details": "no certificate in the chain is revoked"
  },
  "ca_constraints": {
    "ok": true,
    "details": "intermediate and root certificates are CA certificates, leaf is not"
  },
  "certificate_signatures": {
    "ok": true,
    "details": "all certificate signatures were successfully verified"
  },
  "usage": {
    "ok": true,
    "details": "leaf certificate allows digital_signature"
  },
  "identity_match": {
    "ok": true,
    "details": "leaf subject matches expected identity Prof. Mario Rossi"
  },
  "document_signature": {
    "ok": false,
    "details": "document signature could not be verified using the leaf public key"
  }
}

```

5.5.8 Implementation hint for students

A simple implementation is sufficient.

The program may:

1. load the trust store, chain, validation request, revocation list, and, when required, the signed document;
2. identify the first certificate in the chain as the end-entity certificate;
3. identify the last certificate in the chain as the root certificate;
4. evaluate the chain structure and trust checks one by one;
5. reconstruct the certificate payload to be verified for each certificate;
6. verify each certificate signature using the public key of its issuer;
7. depending on the validation mode:

- in `service_identity` mode, verify usage and expected identity;
 - in `document_signature` mode, verify usage, expected identity, and the document signature;
8. record a boolean result and a short explanation string for each check;
 9. combine the results into the final decision.